Research Topics in Blockchain
&
Blockchain and Its Application



Lecture # 9:
Decentralization and Consensus Mechanism

Recap: Bitcoin consensus

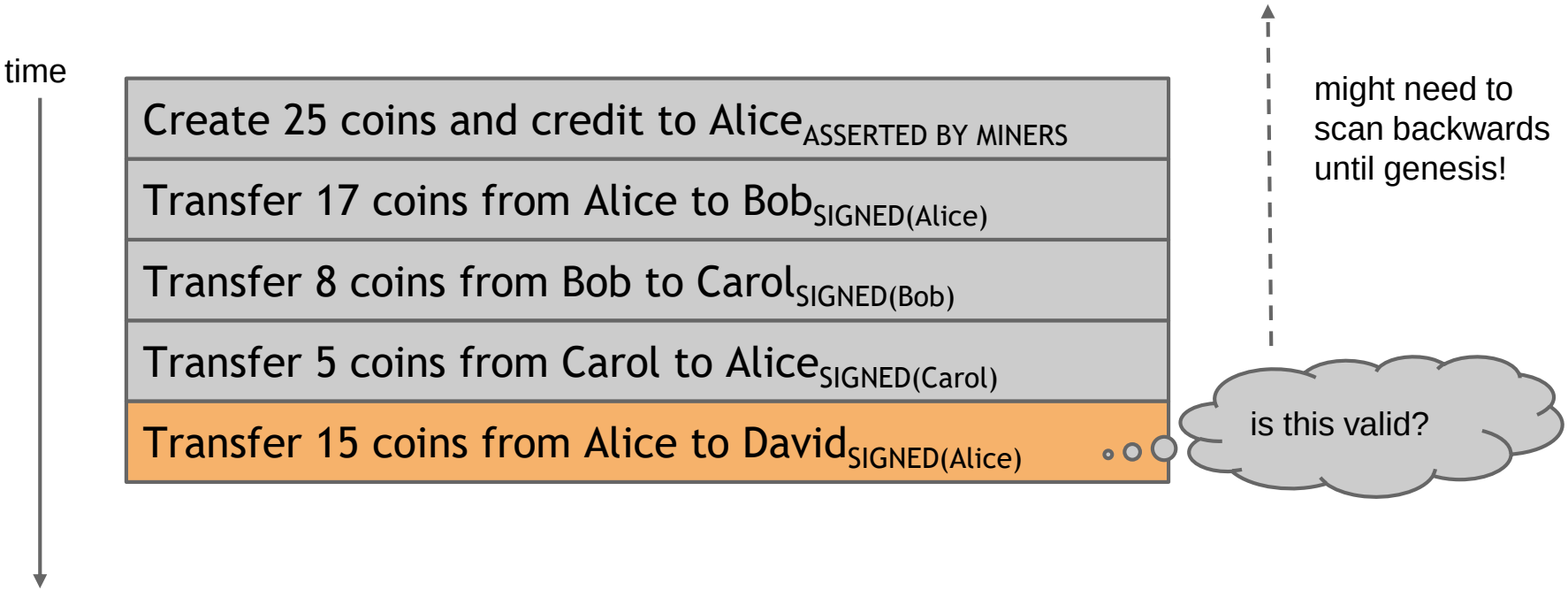
Bitcoin consensus gives us:

- Append-only ledger
- Decentralized consensus
- Miners to validate transactions

assuming a currency exists to motivate miners!

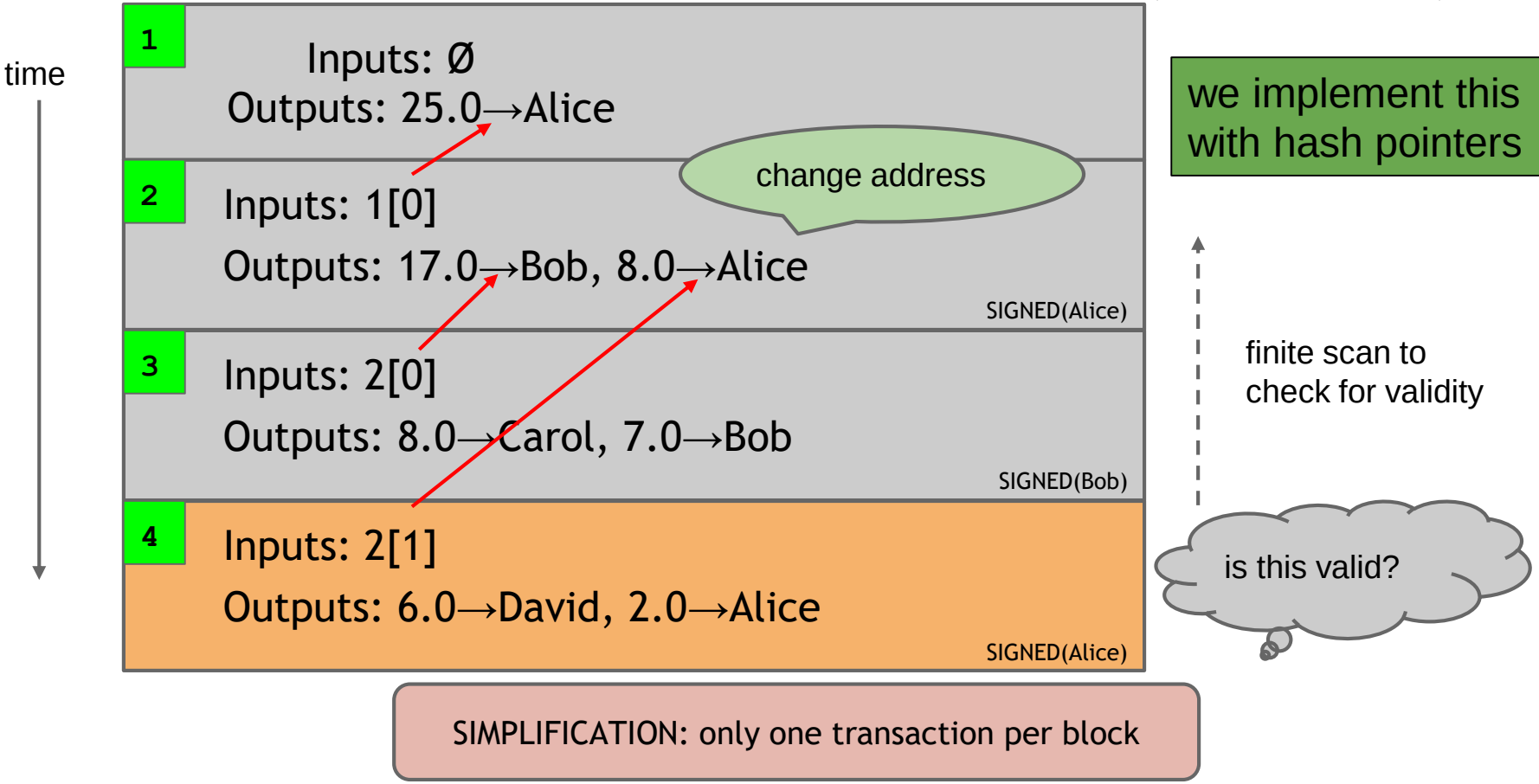
Bitcoin transactions

An account-based ledger (*not* Bitcoin)

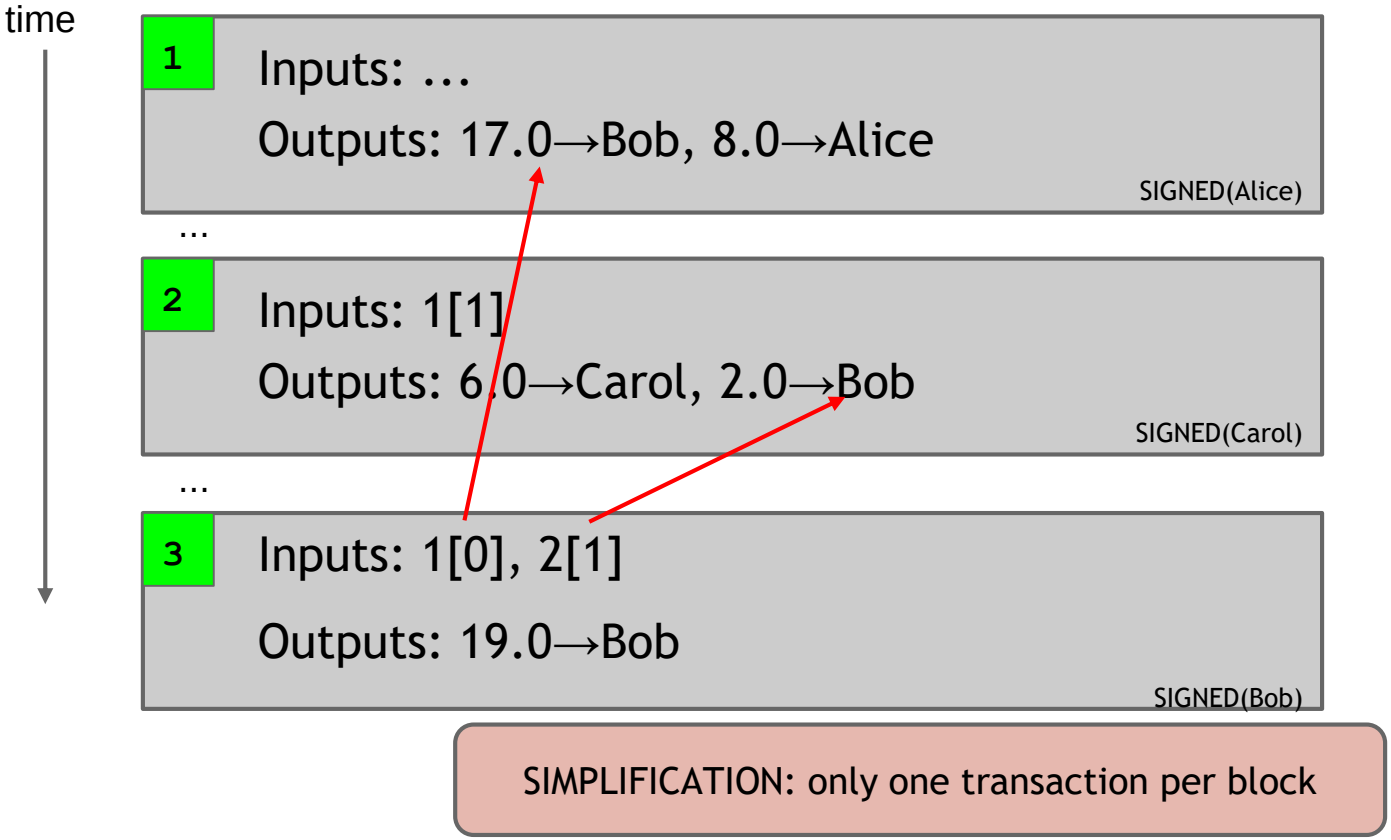


SIMPLIFICATION: only one transaction per block

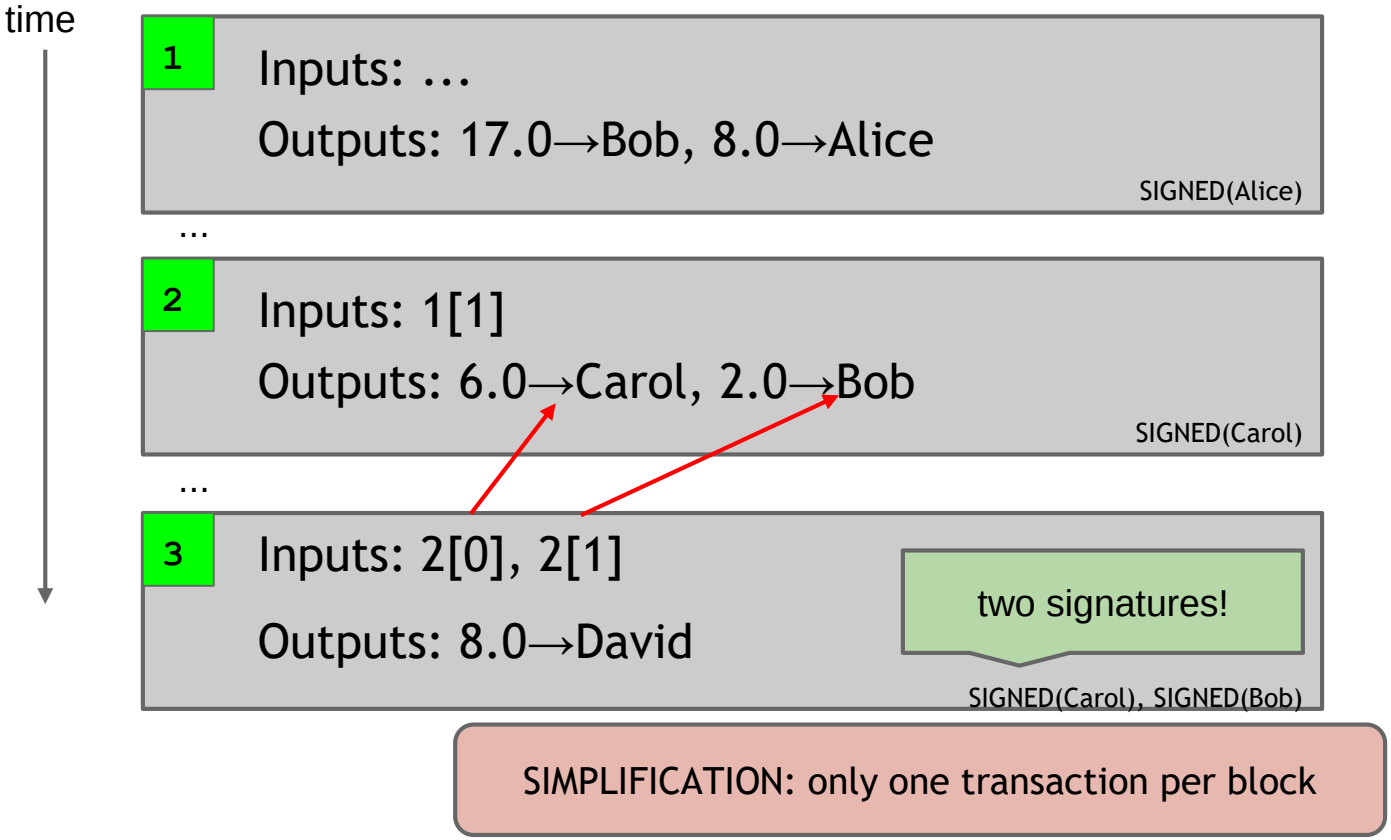
A transaction-based ledger (Bitcoin)



Merging value



Joint payments



Bitcoin Script

What is Bitcoin Script?

✓ Bitcoin Script is a **stack-based, Forth-like programming language** used in Bitcoin transactions to define **spending conditions**.

✓ It is **not Turing-complete** (no loops or recursion) for security reasons.
Bitcoin Script is primarily used in:

Locking scripts (scriptPubKey): Define conditions for spending an output.

Unlocking scripts (scriptSig): Provide proof to satisfy the conditions**.**

How Bitcoin Script Works

Each Bitcoin transaction consists of **inputs and outputs**:

1.Locking Script (scriptPubKey)

- Encodes spending conditions.
- Example: "Only the owner of a private key can spend this output.“

2.Unlocking Script (scriptSig)

- Provides the required proof.
- Example: "Here’s my signature to prove I own the private key."

When a transaction is verified, **Bitcoin executes the script by combining scriptSig and scriptPubKey** in a stack-based execution model.

Locking Script (scriptPubKey)

The sender sets this script to require a valid signature:
plaintext

OP_DUP OP_HASH160 <PublicKeyHash> OP_EQUALVERIFY OP_CHECKSIG

- ✓ OP_DUP → Duplicates the public key.
- ✓ OP_HASH160 → Hashes the public key (RIPEMD160(SHA256(PubKey))).
- ✓ <PublicKeyHash> → The recipient's Bitcoin address.
- ✓ OP_EQUALVERIFY → Verifies the hash matches.
- ✓ OP_CHECKSIG → Checks if the provided signature is valid.

UnLocking Script (scriptSig**)**

The recipient provides a **digital signature** and **public key** to prove ownership:

<Signature> <PublicKey>

Execution Process (Stack-Based Execution)

1.Unlocking Script (scriptSig) is pushed onto the stack:
Plaintext

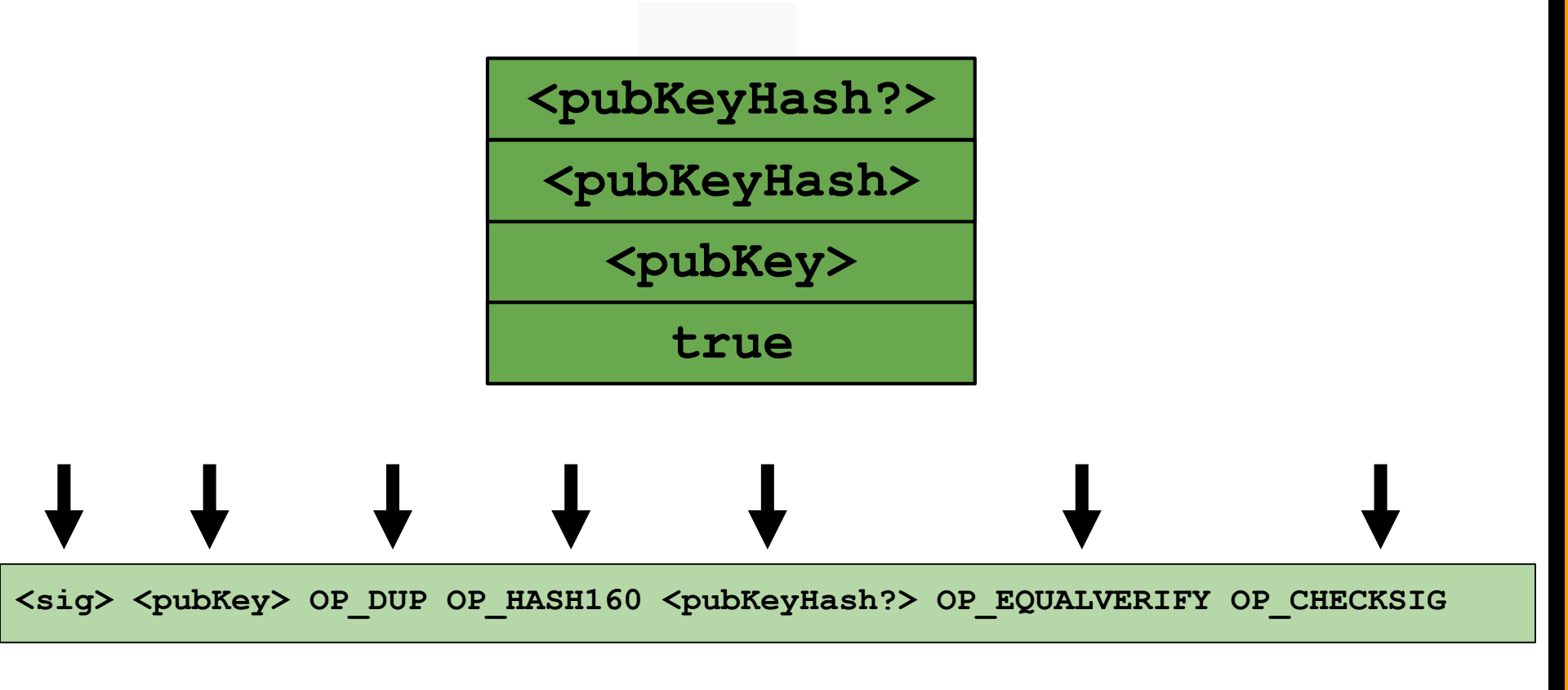
Stack: [Signature, PublicKey]

2.Locking Script (scriptPubKey) executes:

- OP_DUP → Duplicates PublicKey
- OP_HASH160 → Computes RIPEMD160(SHA256(PublicKey))
- OP_EQUALVERIFY → Checks if hash matches <PublicKeyHash>
- OP_CHECKSIG → Verifies the signature

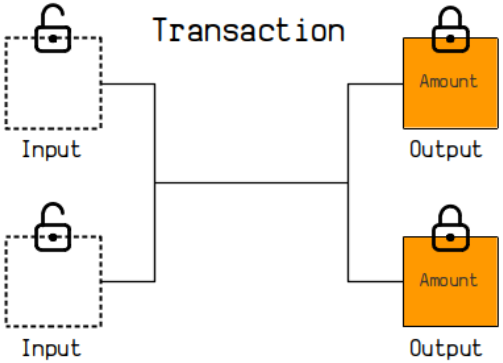
If everything is valid, the script evaluates to **true**, allowing the transaction

Bitcoin script execution example



Opcode	Description	Example (Dummy Values)
`OP_0` / `OP_FALSE`	Pushes an empty value (false) onto the stack.	`OP_0` → `Stack: []`
`OP_1` to `OP_16`	Pushes numbers 1 to 16 onto the stack.	`OP_2 OP_3 OP_ADD` → `Stack: [5]`
`OP_DUP`	Duplicates the top stack item.	`OP_2 OP_DUP` → `Stack: [2, 2]`
`OP_HASH160`	Hashes the top item using `RIPEMD160(SHA256(x))`.	`OP_DUP OP_HASH160 <pubKeyHash>`
`OP_EQUAL`	Checks if the top two stack items are equal.	`OP_5 OP_5 OP_EQUAL` → `Stack: [True]`
`OP_EQUALVERIFY`	Same as `OP_EQUAL`, but fails if false.	`OP_DUP OP_HASH160 <pubKeyHash> OP_EQUALVERIFY`
`OP_CHECKSIG`	Verifies a digital signature.	`<sig> <pubKey> OP_CHECKSIG`

Transaction



A bitcoin *transaction* is just a bunch of data that unlocks and locks up batches of bitcoins.

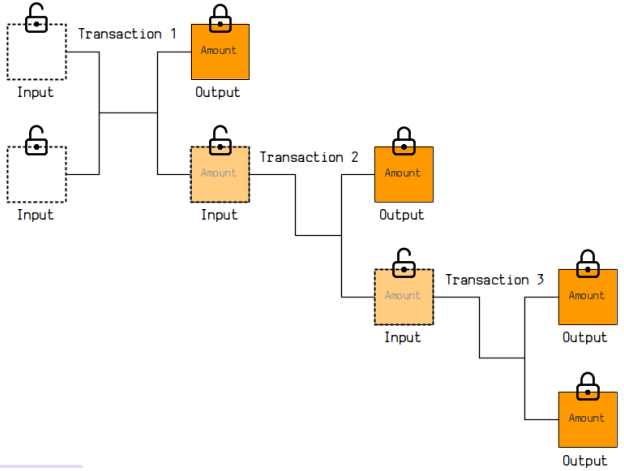
To be more precise, a transaction:

1.Selects existing batches of bitcoins (inputs) and unlocks them.

2.Creates new batches of bitcoins (outputs) and puts new locks on them.

You can have multiple inputs and outputs in a single transaction.

So you can think of a transaction as being **part of a chain** of outputs; one transaction creates an output, and then a future transaction selects that output (as an input) and unlocks it to create new outputs.



{ "version": "02000000", "marker": "00", "flag": "01", "inputcount": "01", "inputs": [{ "txid": "8f1abd8faa06670e451332a5c4861bcdeb768c66bcb66ec419216a5ecbcbe31d", "vout": "01000000", "scriptsigsize": "00", "scriptsig": "", "sequence": "00000080" }], "outputcount": "02", "outputs": [{ "amount": "c747030000000000", "scriptpubkeysize": "17", "scriptpubkey": "a91411b213048648328f84abf9e8f9730328022942b887" }, { "amount": "5d5f830000000000", "scriptpubkeysize": "16", "scriptpubkey": "00145d0d45977f4449b555d795bf76704bee30dba516" }], "witness": [{ "stackitems": "02", "0": { "size": "47", "item": "304402200da87186d2764b38aac32a146597d91f60a7aefa2cb55908c376c91009f20a51022062ffa59a5278a41e674d91de5130e7b9bd50ad8be7de054b807712f6d076b2b401" }, "1": { "size": "21", "item": "03d0e43d864647d4de9eaaa7d5b81c5ca3d2919df554d59951343fe155786c6e5b" } }], "locktime": "00000000" }

The real deal: a Bitcoin transaction

metadata

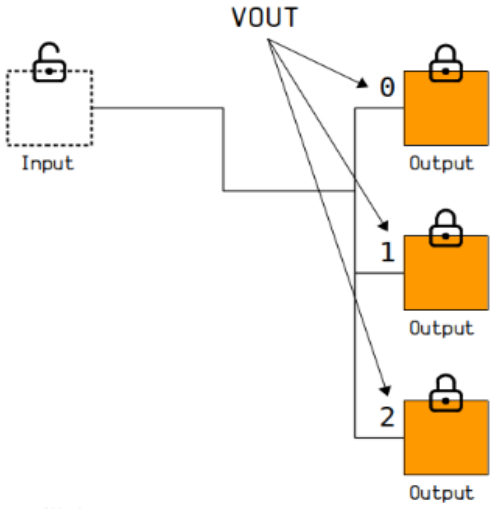
input(s)

output(s)


```
{
    "hash":"5a42590fbe0a90ee8e8747244d6c84f0db1a3a24e8f1b95b10c9e050990b8b6b",
    "ver":1,
    "vin_sz":2,
    "vout_sz":1,
    "lock_time":0,
    "size":404,
    "in":[
     {
       "prev_out":{
         "hash":"3be4ac9728a0823cf5e2deb2e86fc0bd2aa503a91d307b42ba76117d79280260",
         "n":0
       },

          "scriptSig":"30440..."
     },
     {
       "prev_out":{
         "hash":"7508e6ab259b4df0fd5147bab0c949d81473db4518f81afc5c3f52f91ff6b34e",
         "n":0
       },
       "scriptSig":"3f3a4ce81...."
     }
    ],
    "out":[
     {
       "value":"10.12287097",
       "scriptPubKey":"OP_DUP OP_HASH160 69e02e18b5705a05dd6b28ed517716c894b3d42e OP_EQUALVERIFY OP_CHECKSIG"
     }
    ]
}
```

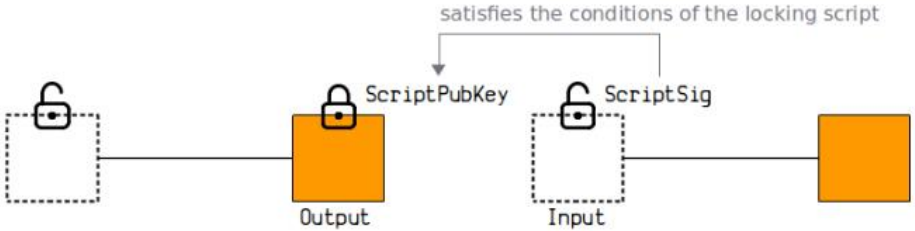
VOUT




```
{
  "version": "01000000",
  "marker": "00",
  "flag": "01",
  "inputcount": "01",
  "inputs": [
    {
      "txid": "9344205e8904b78e4451eadc6120d9304c570f6697d074793c188b7dbdd74cf9",
      "vout": "03000000",
      "scriptsigsize": "17",
      "scriptsig": "160014ceb587f907d67075247ddf52fb343625fd8f5a3b",
      "sequence": "ffffffff"
    }
  ],
  "outputcount": "02",
  "outputs": [
    {
      "amount": "8c6e000000000000",
      "scriptpubkeysize": "19",
      "scriptpubkey": "76a9149e17484e1be011be7587d4f678df1123080fb5bc88ac"
    },
    {
      "amount": "f537010000000000",
      "scriptpubkeysize": "19",
      "scriptpubkey": "76a914c1d3ab2c1c8c9f6b838c9bf25afe1454fbac22fe88ac"
    }
  ],
  "witness": [
    {
```


Finding the transaction that created the output (via its TXID) and then select the specific output you want to spend by using the output's VOUT number.
Every single output in the blockchain can be referenced by using the unique combination of TXID:VOUT. This is referred to as an "outpoint".
For example, here are a few specific outpoints from the blockchain:

ScriptSig



A scriptSig (unlocking script) is used in a Bitcoin transaction to prove ownership and unlock the corresponding **UTXO (Unspent Transaction Output)**.

It is placed in the input section of a transaction and provides:

- ✓ A **digital signature** generated with the sender's private key.
- ✓ The **public key** used to verify the signature.

Full Example

Let's assume **Alice** wants to send **1 BTC** to **Bob**.

Step 1: Alice Creates the Locking Script (`scriptPubKey`)

Alice locks the output with Bob's **hashed public key**:

```
OP_DUP OP_HASH160 76a91488ac OP_EQUALVERIFY OP_CHECKSIG
```

Here, `76a91488ac` represents **Bob's hashed public key**.

Step 2: Bob Wants to Spend the UTXO

Bob must provide a valid `scriptSig` that satisfies `scriptPubKey`.

Bob's `scriptSig` (Unlocking Script):

```
304502210089abcdef123456789abcdefabcdefabcdefabcdefabcd
efabcdefabcdef
2103abcdefabcdefabcdefabcdefabcdefabcdefabcdefabcdefabc
defabcdefabcdef
```

- `3045...` → **Signature** proving Bob owns the private key.
- `2103...` → **Bob's Public Key** for verification.



Step 3: Execution (How Bitcoin Verifies the Transaction)

Bitcoin combines **scriptSig** and **scriptPubKey**, executing them sequentially:

Push `scriptSig` onto stack

```
Stack: [Signature, PublicKey]
```

Execute `scriptPubKey`

- `OP_DUP` → Duplicates `PublicKey`
- `OP_HASH160` → Hashes the top item (`PublicKeyHash_computed`)
- `OP_EQUALVERIFY` → Compares with `76a91488ac` (original `PublicKeyHash`)
- `OP_CHECKSIG` → Verifies `Signature` using `PublicKey`

If valid, transaction is confirmed! ✓

Bitcoin script instructions

256 opcodes total (15 disabled, 75 reserved)

- Arithmetic
- If/then
- Logic/data handling
- Crypto!
  - Hashes
  - Signature verification
  - Multi-signature verification

OP_CHECKMULTISIG

- Built-in support for joint signatures
- Specify *n* public keys
- Specify *t*
- Verification requires *t* signatures



BUG ALERT: Extra data value popped from the stack and ignored

Proof-of-burn

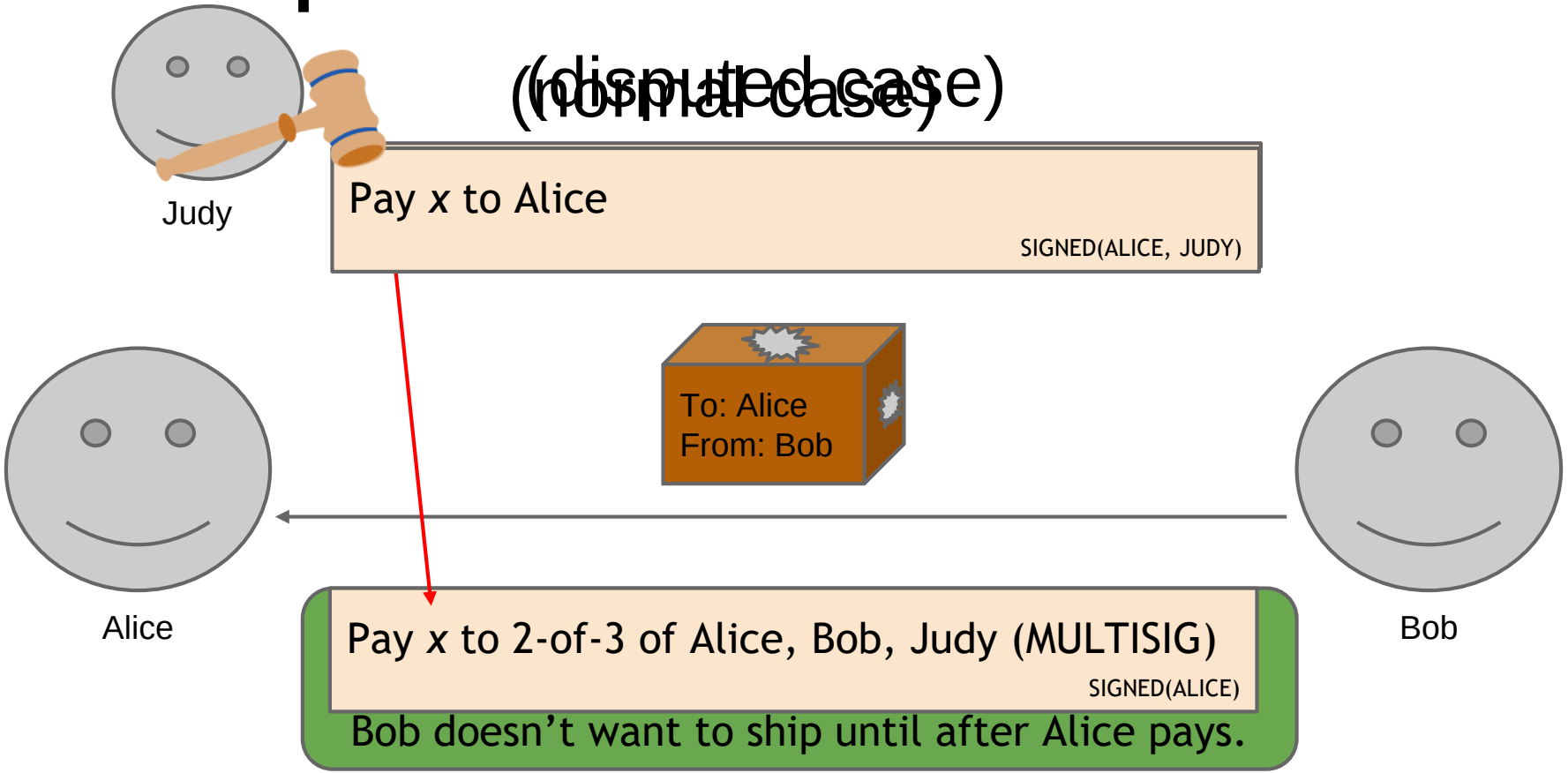
How PoB Works in Bitcoin?

•A user **sends BTC to a burn address** that **no one controls** (e.g., an address without a known private key).

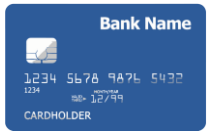
- ✓ This proves that the user has permanently removed the coins from circulation.
- ✓ The transaction is recorded on the blockchain as proof of resource sacrifice.

Applications of Bitcoin scripts

Example 1: Escrow transactions



Example 2: Green addresses



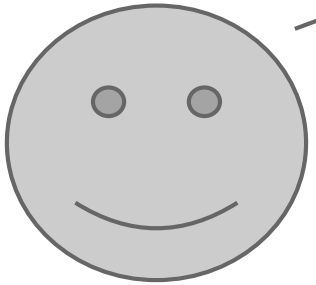
It's me, Alice! Could you make out a green payment to Bob?



Bank

004 days since last double spend!

Faraday cage



Alice

Pay x to Bob, y to Bank

No double spend

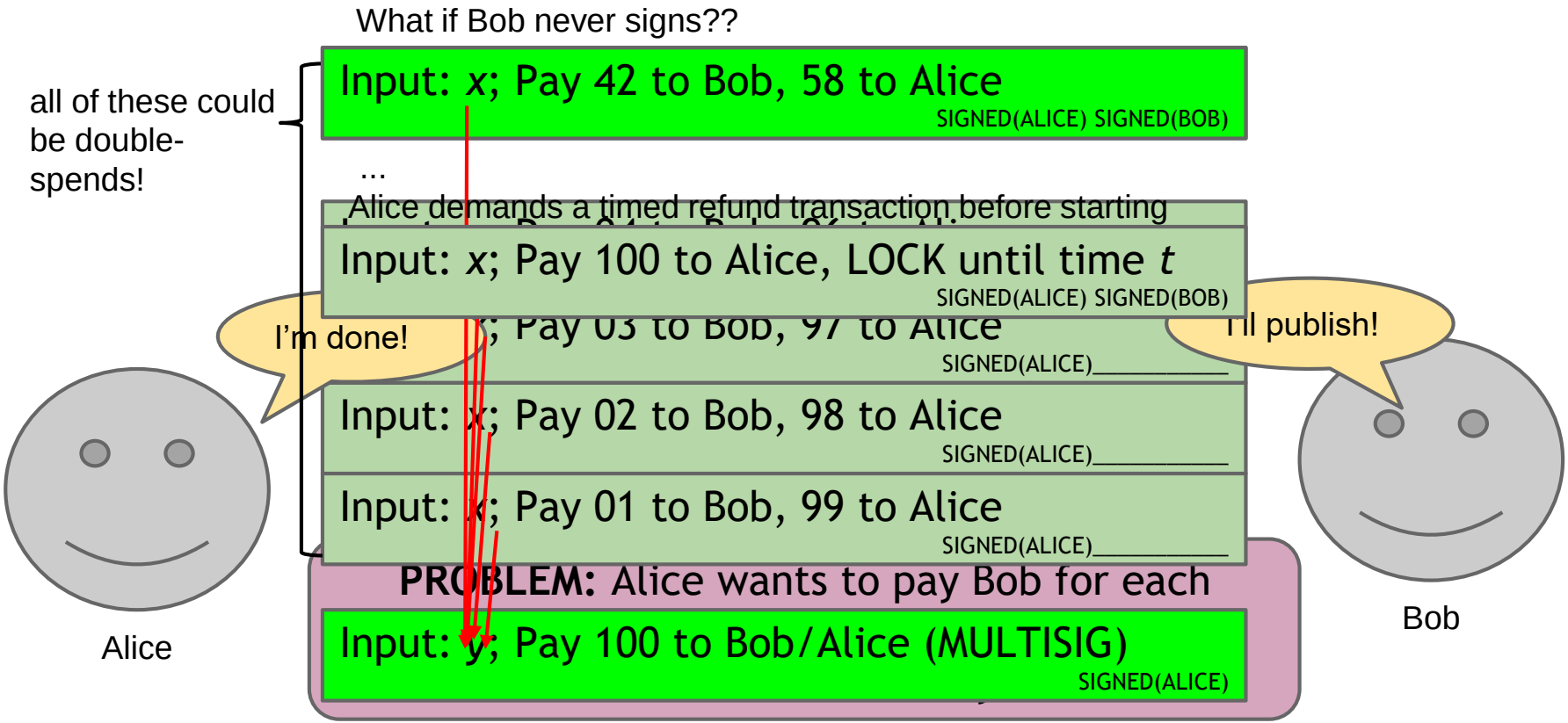
SIGNED(BANK)



Bob

PROBLEM: Alice wants to pay Bob.
Bob can't wait 6 verifications to guard against double-spends, or is offline completely.

Example 3: Efficient micro-payments



lock_time


```
{
    "hash":"5a42590...b8b6b",
      "ver":1,
      "vin_sz":2,
      "vout_sz":1,
      "lock_time":315415,
      "size":404,
...
}
```


Block index or real-world timestamp before which this transaction can't be published

More advanced scripts

- Multiplayer lotteries
- Hash pre-image challenges
- Coin-swapping protocols
    - Don’t miss the lecture on anonymity!

“Smart contracts”

Bitcoin blocks

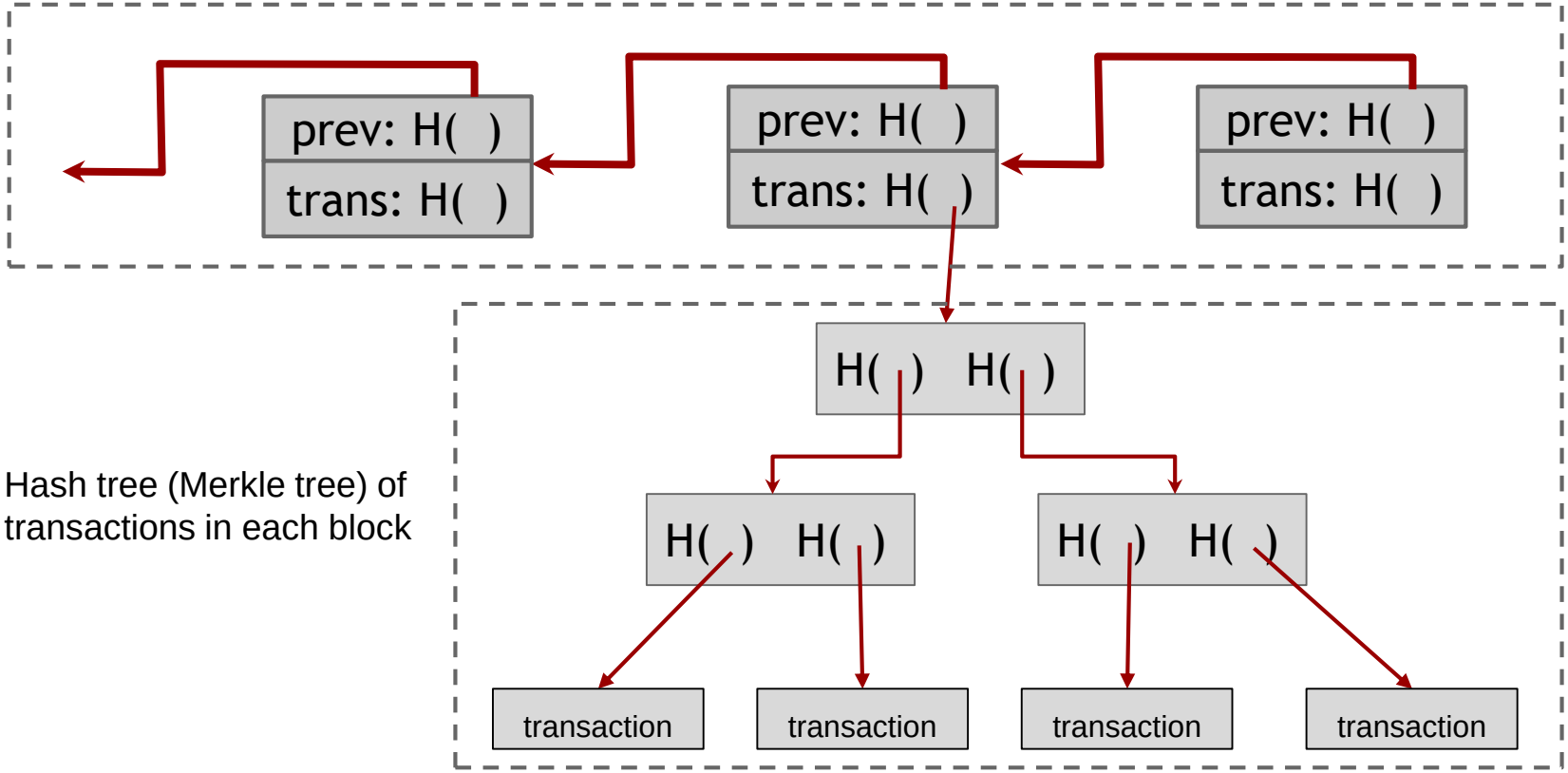
Bitcoin blocks

Why bundle transactions together?

- Single unit of work for miners
- Limit length of hash-chain of blocks
    - Faster to verify history

Bitcoin block structure

Hash chain of blocks



Hash tree (Merkle tree) of transactions in each block

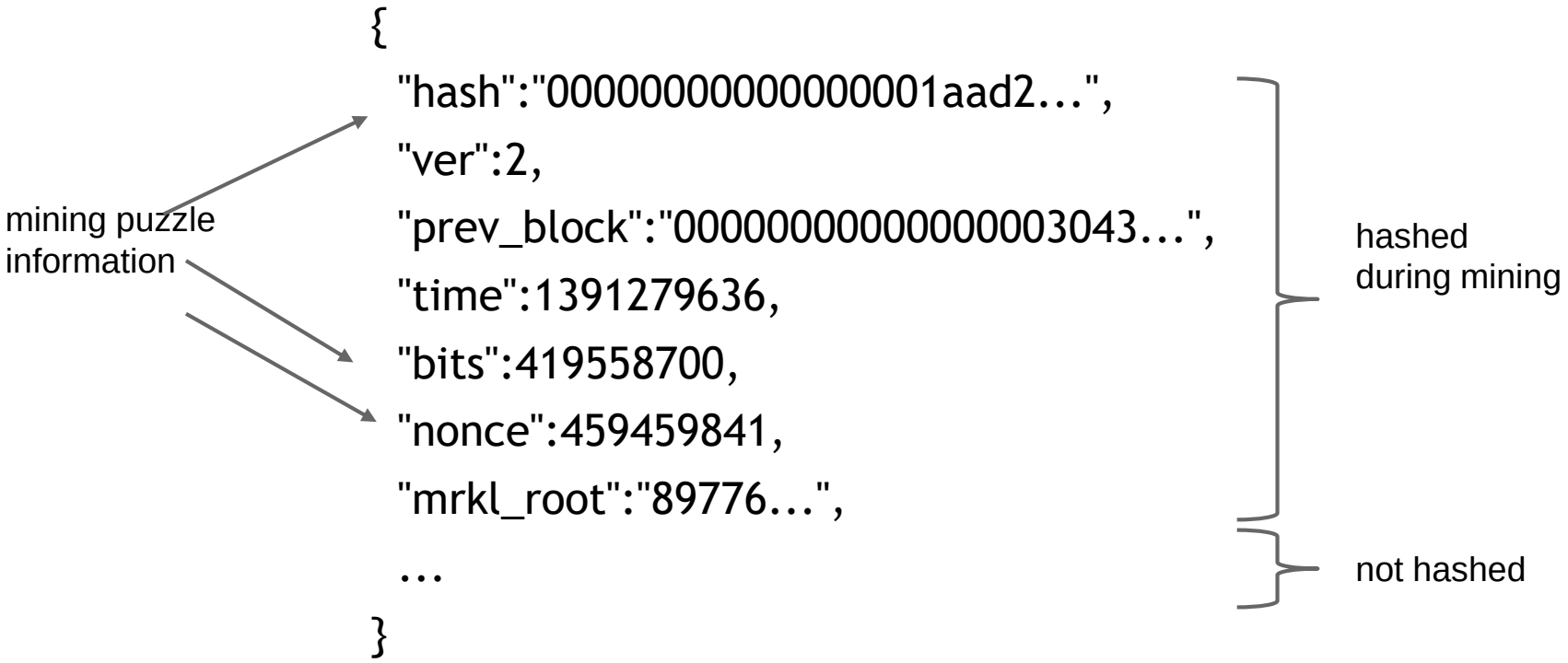
The real deal: a Bitcoin block

block header

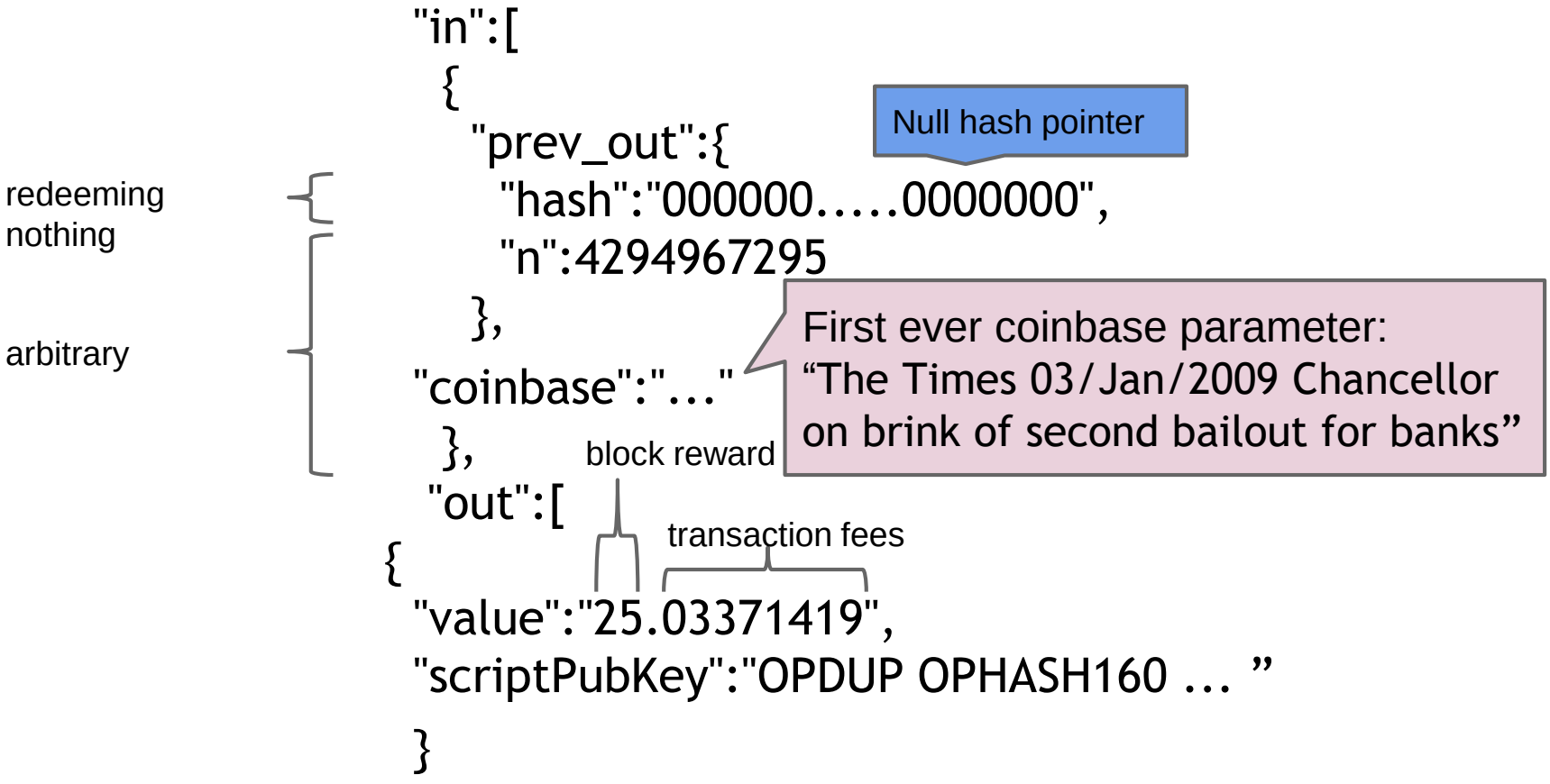
transaction data


```
{
  "hash":"00000000000000001aad2...",
  "ver":2,
  "prev_block":"0000000000000003043...",
  "time":1391279636,
  "bits":419558700,
  "nonce":459459841,
  "mrkl_root":"89776...",
  "n_tx":354,
  "size":181520,
  "tx":[
    ...
  ],
  "mrkl_tree":[
    "6bd5eb25...",
    ...
    "89776cdb..."
  ]
}
```

The real deal: a Bitcoin block header



The real deal: coinbase transaction



See for yourself!

Transaction View information about a bitcoin transaction

151b750d1f13e76d84e82b34b12688811b23a8e3119a1cba4b4810f9b0ef408d

1KryFUt9tXHvaoCYTNPbqpWPJKQ717YmL5	1KvrdrQ3oGqMAiDTMEYCcdDSnVaGNW2YZh	1.0194 BTC
	1KryFUt9tXHvaoCYTNPbqpWPJKQ717YmL5	3.458 BTC

9 Confirmations | 4.4774 BTC

Summary	
Size	257 (bytes)
Received Time	2014-08-05 01:55:25
Included In Blocks	314018 (2014-08-05 02:00:40 +5 minutes)
Confirmations	9 Confirmations
Relayed by IP	Blockchain.info
Visualize	View Tree Chart

Inputs and Outputs	
Total Input	4.4775 BTC
Total Output	4.4774 BTC
Fees	0.0001 BTC
Estimated BTC Transacted	1.0194 BTC
Scripts	Show scripts & coinbase

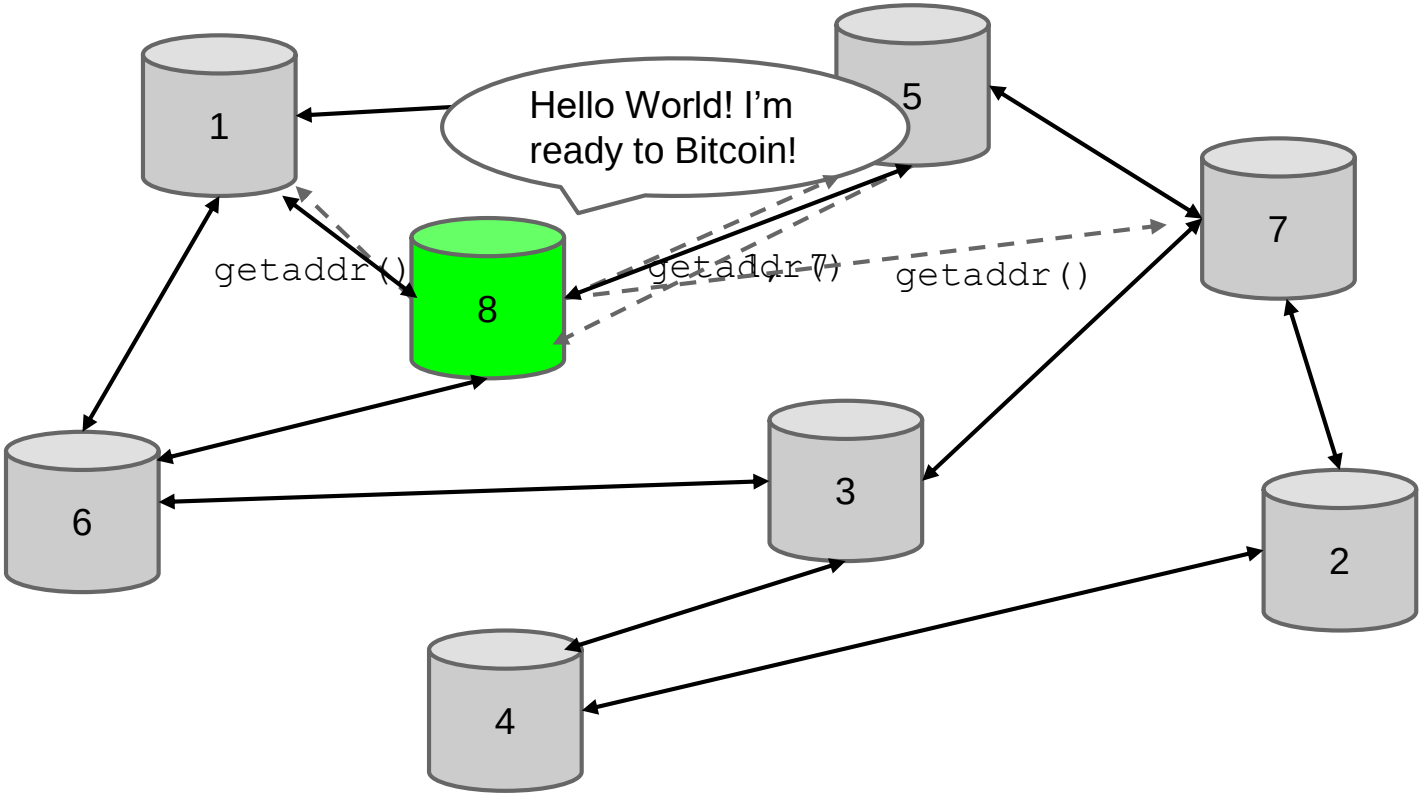
blockchain.info (and many other sites)

The Bitcoin network

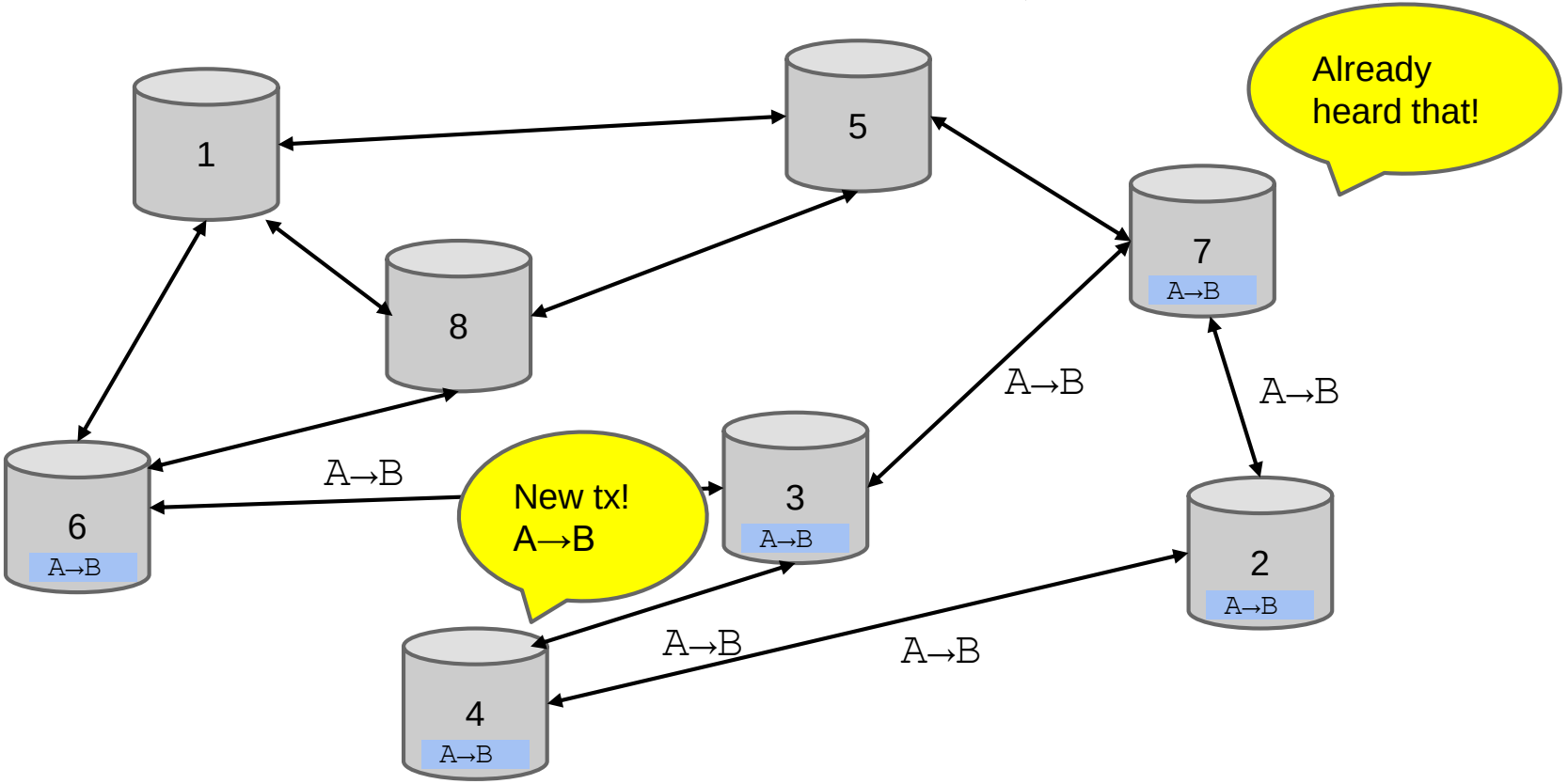
Bitcoin P2P network

- Ad-hoc protocol (runs on TCP port 8333)
- Ad-hoc network with random topology
- All nodes are equal
- New nodes can join at any time
- Forget non-responding nodes after 3 hr

Joining the Bitcoin P2P network

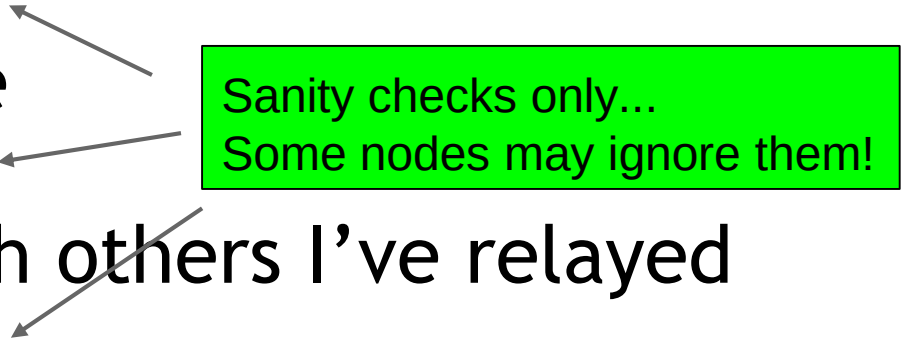


Transaction propagation (flooding)

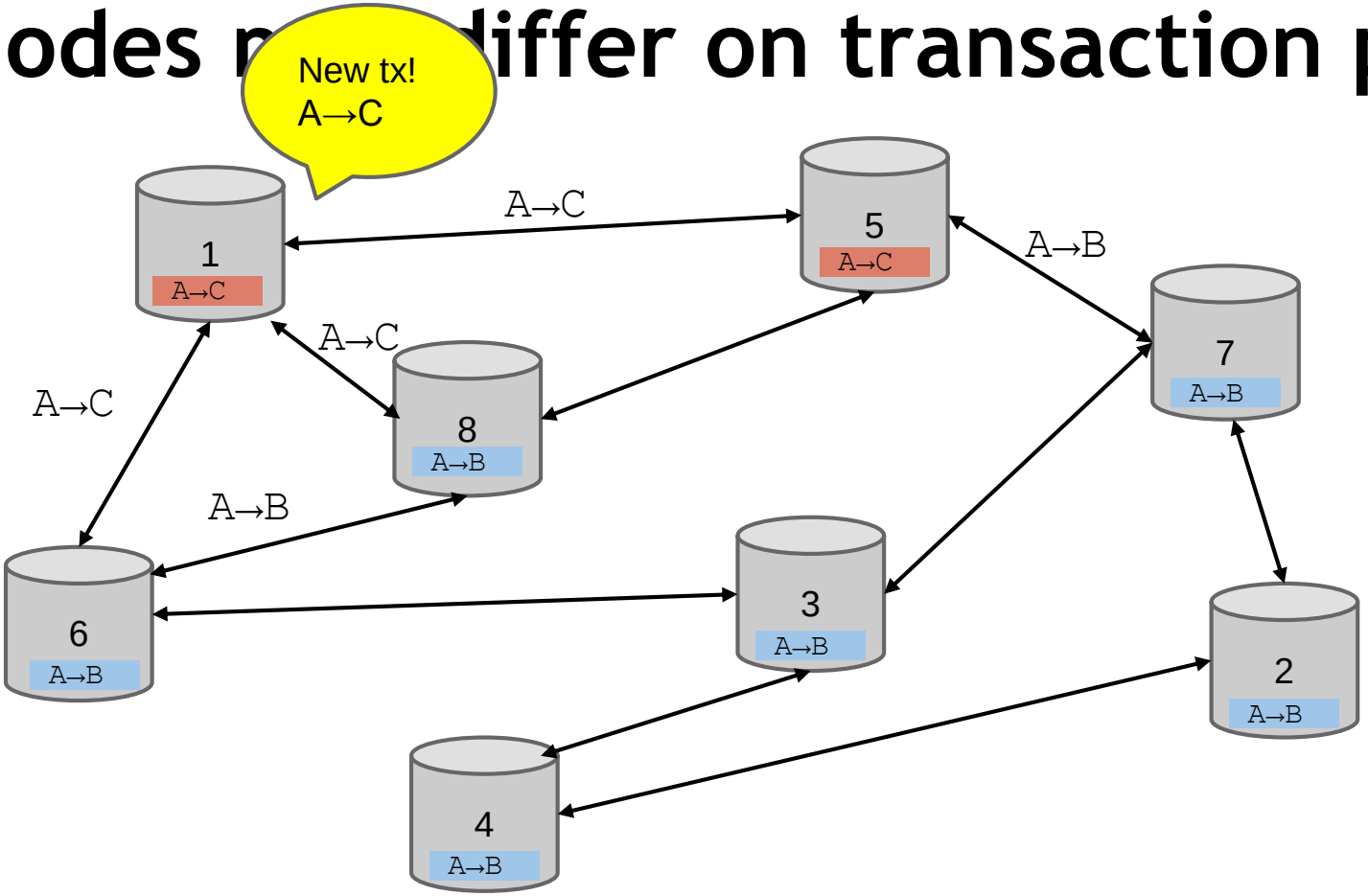


Should I relay a proposed transaction?

- Transaction valid with current block chain
- (default) script matches a whitelist
  - Avoid unusual scripts
- Haven't seen before
  - Avoid infinite loops
- Doesn't conflict with others I've relayed
  - Avoid double-spends



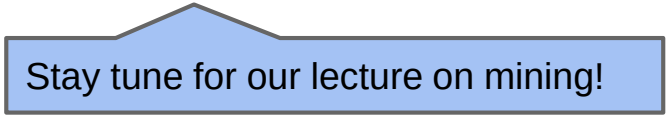
Nodes may differ on transaction pool



Race conditions

Transactions or blocks may *conflict*

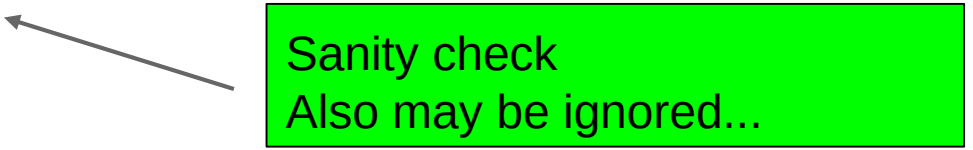
- Default behavior: accept what you hear first
- Network position matters
- Miners may implement other logic!

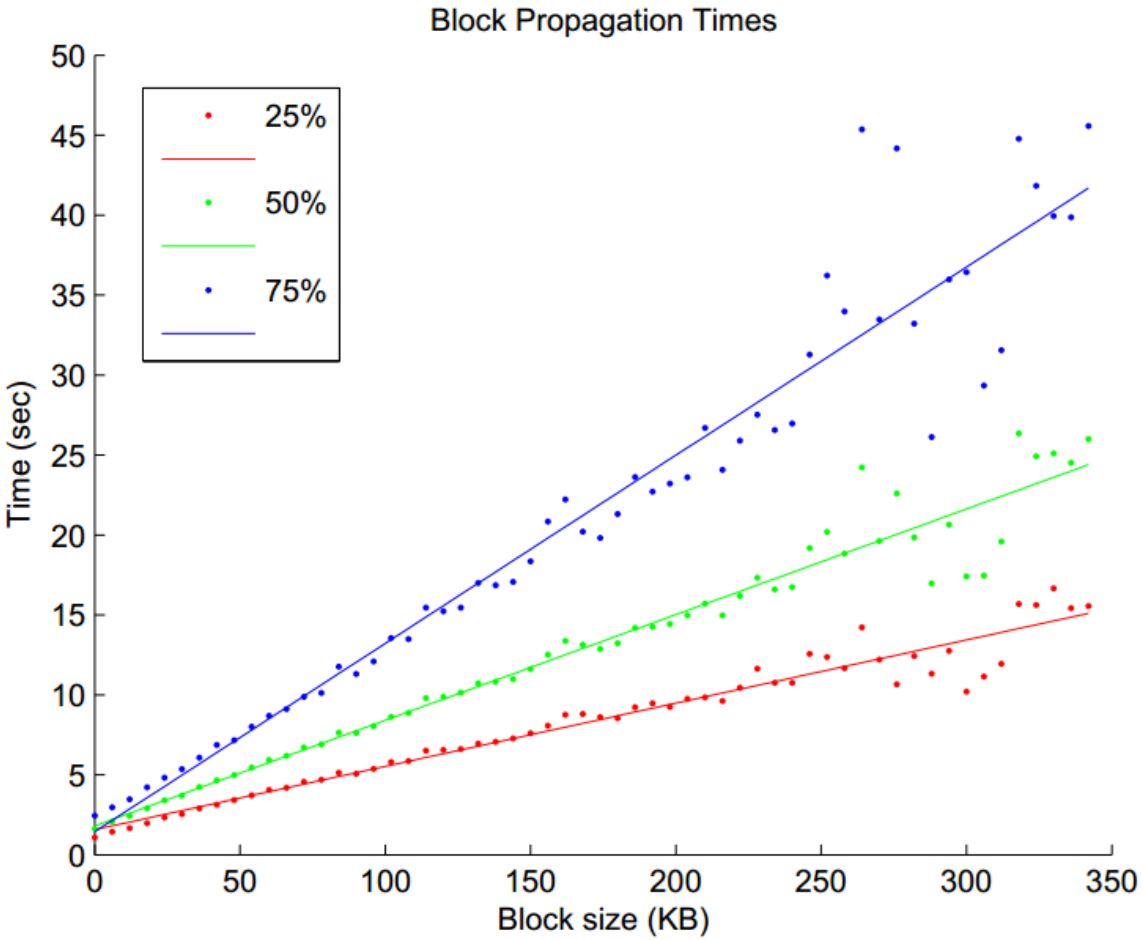


Block propagation nearly identical

Relay a new block when you hear it if:

- Block meets the hash target
- Block has all valid transactions
  - Run *all* scripts, even if you wouldn't relay
- Block builds on current longest chain
  - Avoid forks





Source: Yonatan Sompolinsky and Aviv Zohar: “Accelerating Bitcoin’s Transaction Processing” 2014

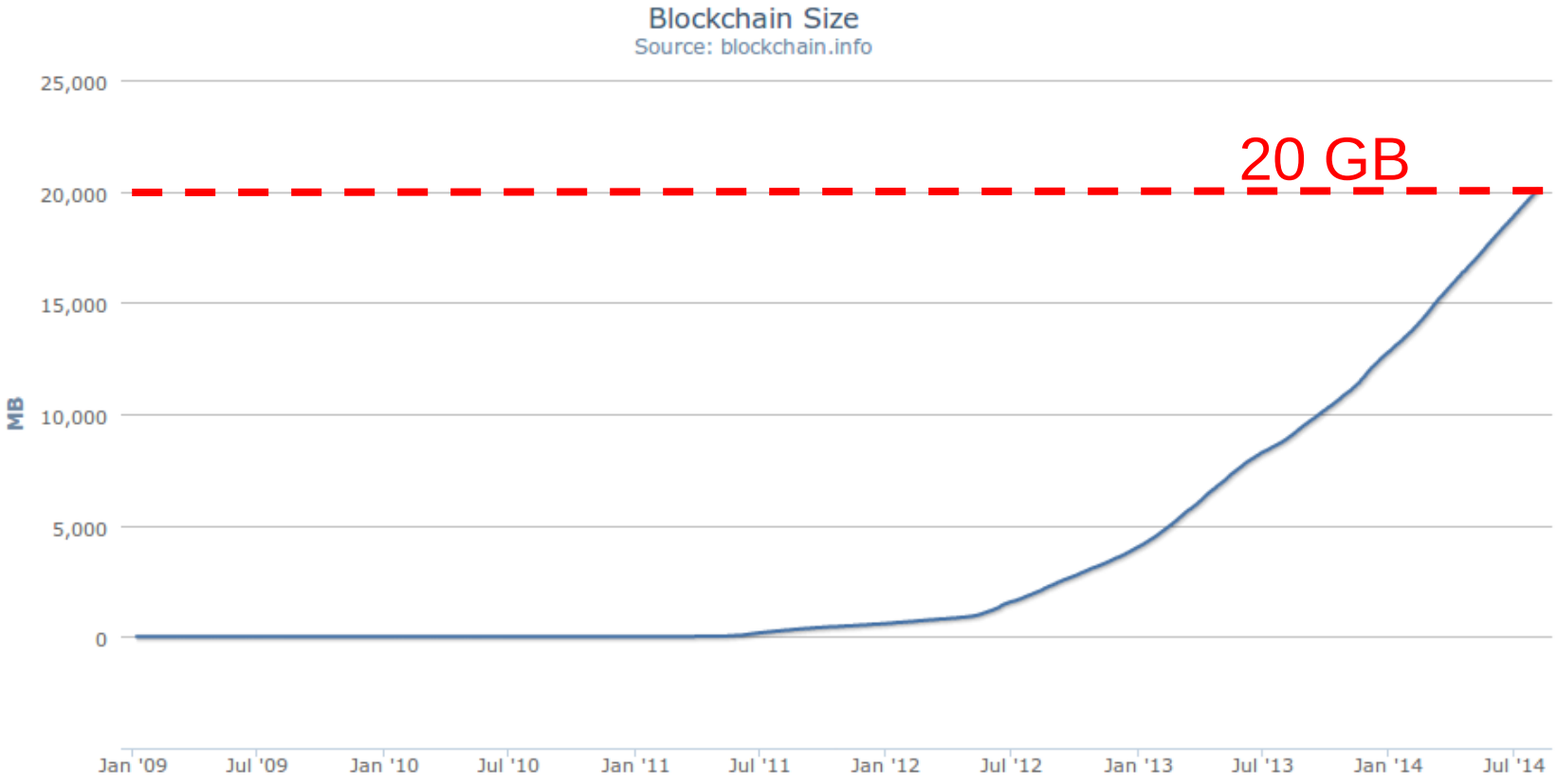
How big is the network?

- Impossible to measure exactly
- Estimates-up to 1M IP addresses/month
- Only about 5-10k “full nodes”
    - Permanently connected
    - Fully-validate
- This number may be dropping!

Fully-validating nodes

- Permanently connected
- Store entire block chain
- Hear and forward every node/transaction

Storage costs



Tracking the UTXO set

- **U**nspent **T**ransaction **O**utput
  - Everything else can be stored on disk
- Currently ~12 M UTXOs
  - Out of 44 M transactions
- Can easily fit into RAM

Thin/SPV clients (not fully-validating)

Idea: don’t store everything

- Store block headers only
- Request transactions as needed
  - To verify incoming payment
- Trust fully-validating nodes

1000x cost savings! (20 GB-23MB)

Software diversity

- About 90% of nodes run “Core Bitcoin” (C++)
  - Some are out of date versions
- Other implementations running successfully
  - BitcoinJ (Java)
  - Libbitcoin (C++)
  - btcd (Go)
- “Original Satoshi client”

Limitations & improvements

Hard-coded limits in Bitcoin

- 10 min. average creation time per block
- 1 M bytes in a block
- 20,000 signature operations per block
- 100 M *satoshis* per bitcoin
- 23M total bitcoins maximum
- 50,25,12.5... bitcoin mining reward



These affect economic balance of power too much to change now

Throughput limits in Bitcoin

- 1 M bytes/block (10 min)
- >250 bytes/transaction
- 7 transactions/sec ☹

Compare to:

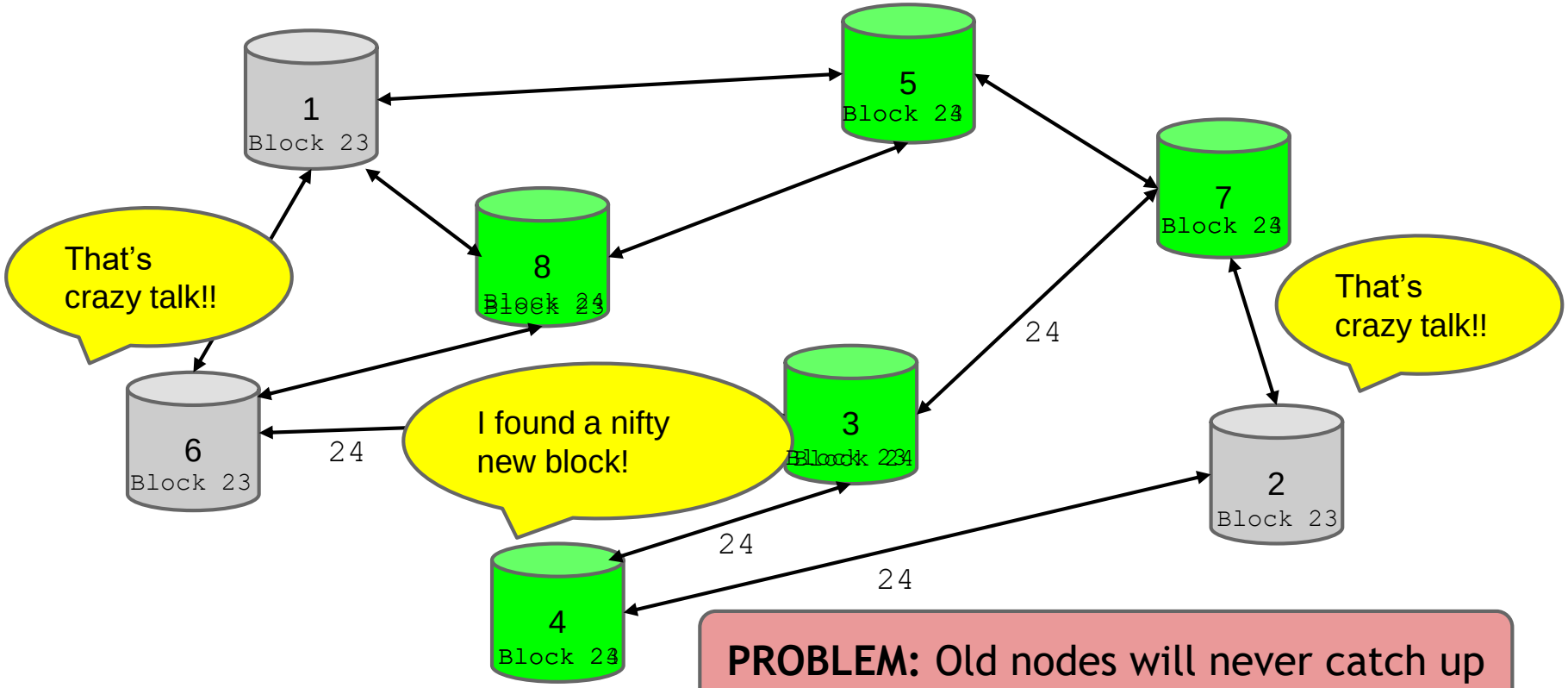
- VISA: 2,000-10,000 transactions/sec
- PayPal: 50-100 transaction/sec

Cryptographic limits in Bitcoin

- Only 1 signature algorithm (ECDSA/P256)
- Hard-coded hash functions

Crypto primitives might break by 2040...

“Hard-forking” changes to Bitcoin



Soft forks

Observation: we can add new features which only *limit* the set of valid transactions

Need majority of nodes to enforce new rules

Old nodes will approve

RISK: Old nodes might mine now-invalid blocks

Soft fork example: pay to script hash

<signature>
<<pubkey> OP_CHECKSIG>

OP_HASH160
<hash of redemption script>
OP_EQUAL

Old nodes will just approve the hash, not run the embedded script

Soft fork possibilities

- New signature schemes
- Extra per-block metadata
  - Shove in the coinbase parameter
  - Commit to UTXO tree in each block

Hard forks

- New op codes
- Changes to size limits
- Changes to mining rate
- Many small bug fixes

Currently seem very unlikely to happen

Stay tuned for our lecture on altcoins!